ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ 1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв: “Linux Kernel Module for Task Information. Programming Project 2 (3-р бүлэг), Task hierarchy”

Хуваарилалтын код: IV-2-2

Хичээл заасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн баг: Б.Төгөлдөр /B242270047/
С.Пүрэвтулга /B241940099/
Л.Пүрэв-очир /B232270108/
Б.Билгүүн /B190910014/
Л.Тэмүүлэн /B210930842/
Ч.Алтангэрэл /B231910003/

Огноо: 2026 оны 10 дугаар сарын 27

Хураангуй

Асуудал. Хэрэглэгчийн орон зайн программ процессын мэдээллийг (PCB буюу `struct task_struct`) шууд уншиж чадахгүй. Сурах бичгийн 3-р бүлгийн Programming Project 2 нь `/proc/pid` виртуал файлаар PID хүлээн авч, тухайн task-ийн нэр, PID, төлөвийг буцаадаг kernel module бичихийг шаарддаг. Багшийн оноосон дүрслэл нь сонгосон процессын эцэг, хүүхдүүдийг шатлалтай мод хэлбэрээр харуулах *task hierarchy* юм.

Шийдэл. `pid.ko` module нь `/proc/pid`-д бичсэн PID-ийг `kstrtol`-оор шалгаж хадгална. Уншихад `seq_file`-ээр номын яг форматтай мөр (`command = [..] pid = [..] state = [..]`), дараа нь `parent` ба `child` мөрүүдийг хэвлэнэ. Python стандарт сан дээрх вэб сервер module-тэй `/proc/pid`-ээр харилцаж, SVG мод, төлөвийн өнгө, өөрчлөлтийн тодруулга, туршилтын процессын модыг харуулна.

Туршилт. Номын жишээ (bash, PID 1), байхгүй PID, 7 төрлийн буруу оролт, kernel thread, хүү процесс үүсгэх тестүүдийг bare-metal Ubuntu 24.04 (kernel 7.0.0) болон хөгжүүлэлтийн WSL2 (kernel 6.6.87.2) дээр ажиллуулсан.

Гол үр дүн. Bash скриптийн 12 тест (bare-metal дээр 12/12; WSL2-д kernel thread тест алгасагдсан), 2-р бүлгийн бэлтгэл module-ийн 9 шалгалт, UI-ийн 24 API ба 12 байрлалын тест бүгд давсан; module ↔ UI интерфейсийн шалгагч 17 ✔, 0 ✖. Туршилтаар PID namespace-ийн алдаа (`task->pid` → `task_pid_vnr`) илэрч засагдсан.

Агуулга

Хураангуй **1**

1 Удиртгал **3**

2 Сурах бичгийн шаардлага **3**

3 Онолын үндэс **4**

3.1 PCB ба `task_struct` . . . 4

3.2 Процессын мод ба төлөв . . . 4

3.3 `/proc` файлын систем . . . 5

3.4 Kernel ба user space-ийн хил . . . 5

3.5 Зэрэгцээ хандалт: RCU ба мутекс . . . 5

3.6 PID namespace . . . 5

4 Системийн зохиомж **5**

5 Хэрэгжилт **6**

5.1 Бэлтгэл module-ууд (`src/prereq/`) . . . 6

5.2 Kernel module — write . . . 6

5.3 Kernel module — read . . . 7

5.4 UI сервер . . . 7

5.5 GitHub дээрх ажиллагаа 8
5.6 Бэлэн код, сан, AI-ийн хэрэглээ 8

6 Туршилт ба UI дүрслэл **9**

7 Туршилтын арга **9**

8 Үр дүн **10**
8.1 Тестийн хураангуй 10
8.2 Bare-metal дээрх бодит гаралт 10
8.3 Гүйцэтгэл (WSL2, 69 процесс, 20 хэмжилтийн медиан) 11
8.4 Screenshot 12

9 Хэлэлцүүлэг **14**

10 Дүгнэлт **15**

Ашигласан материал **15**

Хавсралт А. Ажлын төлөвлөгөө **16**

1. Удиртгал

Зорилго. Linux kernel-ийн доторх процессын өгөгдлийн бүтцийг (`task_struct`, процессын мод) `/proc` файлын системээр дамжуулан хэрэглэгчид гаргаж, түүнийг интерактив мод хэлбэрээр дүрслэх.

Ач холбогдол. Энэ төсөл нь онолын хэд хэдэн ойлголтыг бодитоор холбоно: процесс ба PCB (OSC §3.1), процесс үүсгэх ба процессын мод (§3.3, `fork`, `wait`, zombie), kernel ба user space-ийн хил (`copy_from_user`), kernel module (2-р бүлэг), зэрэгцээ хандалт ба синхрончлол (RCU, мутекс — 6-р бүлэг).

Хамрах хүрээ.

- `src/prereq/` — 2-р бүлгийн бэлтгэл module-ууд: `simple`, `/proc/hello`, `/proc/jiffies`, `/proc/seconds`.
- `src/kmod/pid.c` — үндсэн module: `/proc/pid` write ба read, parent/children өргөтгөл.
- `src/ui/` — task hierarchy вэб UI, туршилтын процессын мод, module ↔ UI интерфейсийн шалгагч.
- `tests/` — үндсэн ба захын тест; `docs/Manual.md` — хэрэглэгчийн ба operational заавар.

Хамрахгүй: kernel-ээс процессын үүсэл/дуусгаврыг эвентээр мэдээлэх (UI тогтмол хугацаанд асуудаг), `/proc/pid`-ийг олон хэрэглэгч зэрэг ашиглах.

2. Сурах бичгийн шаардлага

OSC (2018), 3-р бүлэг, Programming Project 2 “Linux Kernel Module for Task Information” (P-15–P-17):

1. `/proc/pid` файл үүсгэх module бичих.
2. Хэрэглэгч `echo "1395" > /proc/pid` гэж PID бичнэ. Module оролтыг `kmalloc` + `copy_from_user`-ээр kernel санах ой руу хуулж, `kstrtol`-оор тоо болгон хадгална.
3. `cat /proc/pid` хийхэд тухайн task-ийн `command`, `pid`, `state`-ийг хэвлэнэ. Task-ийг `pid_task(find_vpid(pid), PIDTYPE_PID)`-ээр олно.
4. Хүчингүй PID бол `find_vpid` `NULL` буцаах тул юу ч хэвлэхгүй (read 0 байт).

```
$ echo "1395" > /proc/pid
$ cat /proc/pid
command = [bash] pid = [1395] state = [1]
```

Нэмэлт шаардлага (BD1 заавар). Номын үндсэн функцийг бүрэн хэрэгжүүлэх; оролтыг өөрчилж дахин турших; буруу, захын оролтыг шалгах; оноосон дүрслэл бүхий UI-ийг бодит ажиллагаатай холбох; Manual-ийн дагуу өөр компьютерт дахин ажиллуулах боломж бүрдүүлэх.

Хүснэгт 1: Шаардлага ба хэрэгжүүлсэн газар

Шаардлага	Хэрэгжилт	Төлөв
`/proc/pid` үүсгэх	`proc_create("pid", 0666, ...)`, `struct proc_ops` (`src/kmod/pid.c`)	Хэрэгжсэн
PID бичих (write)	`kmalloc` + `copy_from_user` + `kstrtol`, 1..`PID_MAX_LIMIT` шалгалт	Хэрэгжсэн
command, pid, state хэвлэх (read)	`pid_task(find_vpid())`, `seq_printf`, номын яг формат	Хэрэгжсэн
Хүчингүй PID → 0 байт	`task == NULL` үед юу ч хэвлэхгүй	Хэрэгжсэн
Parent, child (task hierarchy)	`real_parent`, `children` жагсаалт (`parent =`, `child =` мөр)	Нэмэлт
Өөр PID, тохиргоогоор турших	UI-ийн PID талбар, гүн 0–4, demo мод, автомат шинэчлэлт	Хэрэгжсэн
Буруу, захын оролт	`-EINVAL`/`-ERANGE`; UI улаан мэдэгдэл; `tests/boundary` (7)	Хэрэгжсэн
Task hierarchy UI	`src/ui/`: SVG мод, төлөвийн өнгө, түүх, reset	Хэрэгжсэн
Manual, дахин ажиллуулах	`docs/Manual.md` §A/§B, README Quick Start	Хэрэгжсэн

3. Онолын үндэс

3.1 PCB ба `task_struct`

Kernel процесс бүрийг *process control block* буюу Linux-д `struct task_struct`-ээр (`<linux/sched.h>`) төлөөлнө. Энэ төсөлд ашигласан талбарууд:

Талбар	Утга
`comm`	Гүйцэтгэх файлын нэр (16 байт хүртэл)
`__state`	Төлөвийн битийн маск: 0 — running, 1 — `TASK_INTERRUPTIBLE`, 2 — `TASK_UNINTERRUPTIBLE`, 4 — `__TASK_STOPPED`; kernel 5.14-өөс `state`-ийн оронд
`real_parent`	Процессыг `fork` хийсэн эцэг (эцэг нь дуусвал init/subreaper руу шилжинэ)
`children, sibling`	Хүүхдүүдийн давхар холбоост жагсаалт: эцгийн `children` толгой, хүүхэд бүрийн `sibling` зангилаа
`pid, tgid`	Урсгалын ба процессын (thread group) дугаар — *global* namespace-д

3.2 Процессын мод ба төлөв

Linux-д процесс бүр (PID 0-ээс бусад) нэг эцэгтэй тул бүх процесс нэг мод үүсгэнэ: үндэс нь `systemd`/`init` (PID 1). Хүүхэд дуусахад эцэг нь `wait()` хийх хүртэл *zombie* (`Z`) төлөвт үлдэнэ; эцэг нь эхэлж дуусвал хүүхэд *өнчин* болж init руу шилжинэ (OSC §3.3.2). `ps`-ийн үсгэн төлөв (`R S D T Z I`) нь `__state` ба `exit_state`-ээс `task_state_to_char()`-аар гарна.

3.3 `/proc` файлын систем

`/proc` нь дискэн дээр байхгүй, санах ойд л байдаг виртуал файлын систем. `proc_create(name, mode, parent, &proc_ops)` нь файл үүсгэж, унших/бичих бүрт module-ийн функцийг дуудна. Kernel 5.6-аас `struct file_operations`-ийн оронд `struct proc_ops` ашиглана. Урт гаралтыг `seq_file` (`single_open`, `seq_printf`, `seq_read`) буферлэж, offset ба дахин уншилтыг зөв зохицуулна.

3.4 Kernel ба user space-ийн хил

User space-ийн заагч kernel-д хүчингүй эсвэл хортой байж болох тул шууд хандахгүй: `copy_from_user()` хаягийг шалгаж хуулна. Kernel санах ойг `kmalloc(…, GFP_KERNEL)`-ээр авч, бүх замд `kfree()` хийнэ.

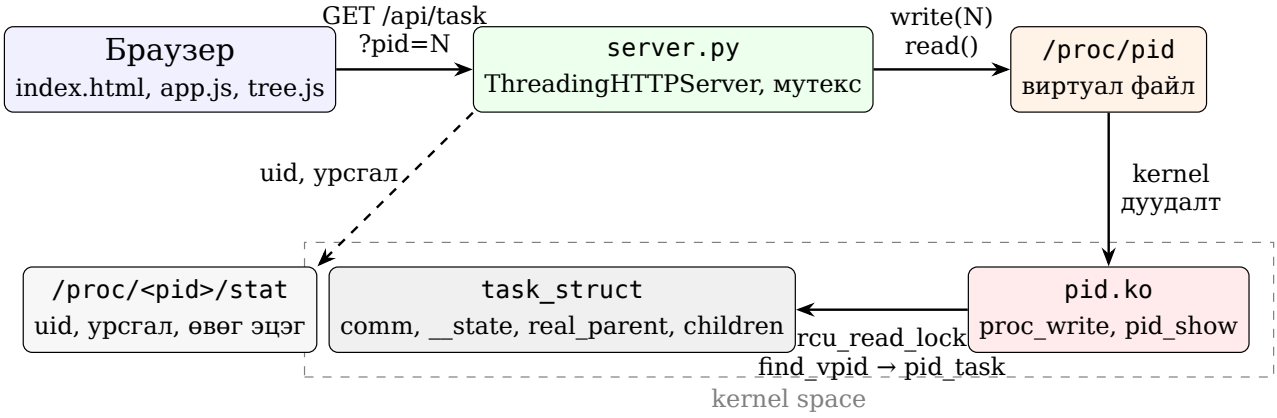
3.5 Зэрэгцээ хандалт: RCU ба мутекс

Процессын жагсаалт байнга өөрчлөгддөг. `find_vpid`, `pid_task`, `real_parent`, `children`-ийг `rcu_read_lock()` дор уншиж, уншиж байх хооронд task чөлөөлөгдөхөөс сэргийлнэ (read-copy-update, OSC §6.7-ийн уншигч/бичигчийн хувилбар). Module сүүлд бичсэн PID-ийг нэг глобал `l_pid`-д хадгалдаг тул “бичих → унших” нь хоёр тусдаа систем дуудалт; хоёр процесс зэрэг ажиллавал *уралдааны нөхцөл* үүснэ. UI сервер үүнийг эгзэгтэй муж болгон мутексээр хамгаална (§5.3).

3.6 PID namespace

Контейнер, WSL2-д процесс өөрийн PID namespace-тэй байж, нэг task өөр namespace-д өөр дугаартай байна. `find_vpid(n)` нь *дуудагчийн* namespace-д хайдаг тул хэвлэх PID-ийг мөн дуудагчийн namespace-ээр (`task_pid_vnr`) гаргах хэрэгтэй.

4. Системийн зохиомж



Зураг 1: Компонент ба өгөгдлийн урсгал. Тасархай — procfs-оос нэмэлт мэдээлэл.

Компонент	Үүрэг
`pid.ko`	`/proc/pid` үүсгэнэ; write — PID шалгаж хадгална; read — task, эцэг, хүүхдүүдийг хэвлэнэ
`server.py`	HTTP + JSON API; оролт шалгах (зөвхөн цифр, 0–4194304); write+read-ийг мутекс дотор; хүүхэд бүрийг дахин асууж `depth` түвшний мод үүсгэнэ (≤ 400 зангилаа)
Backend горим	`module` (`/proc/pid_tree`-тэй), `book` (зөвхөн `/proc/pid` — манай module), `fallback` (module-гүй, тодорхой тэмдэглэнэ)
`app.js`, `tree.js`	SVG мод (tidy tree, олон навчийг сүлжээ), өөрчлөлт харьцуулах, түүх, туршилт удирдах, экспорт
`demo_tree.py`	Мэдэгдэж буй бүтэцтэй туршилтын процессын мод (S, R, T, Z, 5 урсгалтай); signal-ийг зөвхөн энэ бүлэгт
`contract_check.py`	Module-ийн гаралтыг UI-ийн задлагчтай тулгах 18 шалгалт

Гаралтын формат (module ↔ UI гэрээ). Эхний мөр нь номын яг формат; дараагийн мөрүүд нь hierarchy-ийн өргөтгөл. Байхгүй PID бол 0 байт.

```
command = [bash] pid = [5090] state = [1]
parent = [bash] ppid = [5084]
child = [sleep] pid = [5106] state = [8193]
child = [cat] pid = [5108] state = [0]
```

5. Хэрэгжилт

Хэл ба сан: kernel module — C (Linux kernel API, gcc), UI — Python 3 стандарт сан (`http.server`, `threading`) ба vanilla HTML/CSS/JavaScript (SVG). Гадны сан ашиглаагүй.

5.1 Бэлтгэл module-ууд (`src/prereq/`)

2-р бүлгийн Programming Project: `simple.c` ачаалах/буулгахад `dmesg`-д `GOLDEN_RATIO_PRIME, gcd(3300, 24) = 12, jiffies, HZ; hello.c` — `/proc/hello; jiffies.c` — одоогийн `jiffies; seconds.c` — ачаалснаас хойших секунд `(jiffies - start)/HZ`. Бүгд `proc_ops` + `seq_file` загвартай — үндсэн module-ийн суурь.

5.2 Kernel module — write

```
if (count == 0 || count > MAX_INPUT_LEN)          /* MAX_INPUT_LEN = 32 */
        return -EINVAL;
k_mem = kmalloc(count + 1, GFP_KERNEL);
if (!k_mem)
        return -ENOMEM;
if (copy_from_user(k_mem, usr_buf, count)) {
        kfree(k_mem);
        return -EFAULT;
}
k_mem[count] = '\0';
ret = kstrtol(k_mem, 10, &val);                    /* '\n'-ийг зөвшөөрнө */
kfree(k_mem);
if (ret)
```

```
        return ret;                                     /* abc -> EINVAL, хэт том -> ERANGE */
if (val <= 0 || val > PID_MAX_LIMIT)
        return -EINVAL;
l_pid = val;
return count;
```

Номын skeleton-оос ялгаа: оролтын урт хязгаартай; `sscanf`-ийн оронд хатуу `kstrtol`; 0, сөрөг, `PID_MAX_LIMIT`-ээс их утгыг татгалзана; алдаа гарвал өмнөх PID хэвээр.

5.3 Kernel module — read

```
rcu_read_lock();
task = pid_task(find_vpid(l_pid), PIDTYPE_PID);
if (!task)
        goto out;                                       /* байхгүй PID -> 0 байт */
seq_printf(m, "command = [%s] pid = [%d] state = [%u]\n",
           task->comm, task_pid_vnr(task), READ_ONCE(task->__state));
parent = rcu_dereference(task->real_parent);
if (parent && parent != task)
        seq_printf(m, "parent = [%s] ppid = [%d]\n", parent->comm, task_pid_vnr(parent));
list_for_each_entry_rcu(child, &task->children, sibling)
        seq_printf(m, "child = [%s] pid = [%d] state = [%u]\n",
                   child->comm, task_pid_vnr(child), READ_ONCE(child->__state));
out:
rcu_read_unlock();
```

Номын skeleton (kernel 4.x)	Манай module	Шалтгаан
`struct file_operations`	`struct proc_ops`	kernel ≥ 5.6
`task->state`	`task->__state`	kernel ≥ 5.14
`proc_read` + `completed`	`seq_read` + `single_open`	олон хүүхэд, хэсэгчилсэн унших
`task->pid`	`task_pid_vnr(task)`	PID namespace (§9)

5.4 UI сервер

Мутекс. Module нэг глобал PID-тэй тул хүсэлт бүрийн “write → read”-ийг `threading.Lock`-оор нэг эгзэгтэй муж болгоно; тест: 2 урсгал × 15 зэрэг хүсэлт — 30/30 зөв PID.

Мод байгуулах. `build_tree()` сонгосон PID-ийг асууж, хүүхэд бүрийг `depth` түвшин хүртэл дахин асууна (хүүхэд бүрд 1 module асуулга). `book` горимд module-ийн `child` мөрүүдээс хүүхдийн жагсаалт, `/proc/<pid>/stat`-аас uid, урсгалын тоо, PID 1 хүртэлх өвөг эцгийн гинжийг авна. Module хэвлэсэн PID бичсэнээс өөр бол (global PID) UI анхааруулга гаргаж procfs-оор үргэлжилнэ.

Дүрслэл. Төлөвийн өнгө: R ногоон, S цэнхэр, D улбар шар, T саарал, Z улаан, I цайвар. Шинэчлэлт бүрийг өмнөхтэй харьцуулж шинэ процесс (ногоон), төлөв өөрчлөгдсөн (шар, “S→T”), алга болсон (улаан) гэж тодруулна; 50 snapshot-ийн түүх ◀ ▶. Тасархай зангилааг ⊕-оор газар дээр нь нээнэ; гарын товчоор (↑ ↓ ← →, Enter, +/−) удирдана; JSON/SVG/PNG экспорт.

Туршилтын мод. `demo_tree.py` нь тусдаа процессын бүлэгт `demo_root` → sleeper (2 ач), busy, stopped (T), threads (5 урсгал), zombie (Z) үүсгэнэ. UI-аас fork, zombie, `wait()`, STOP/CONT/TERM — зөвхөн энэ бүлэгт (бусад PID → 403).

5.5 GitHub дээрх ажиллагаа

Код https://github.com/Tugu8/os-bd1-IV-2-2- repository-д байрлана. `main` branch руу шууд push хийгээгүй: гишүүн бүр өөрийн fork эсвэл branch дээр ажиллаж, Pull Request (PR) нээж merge хийсэн (`CONTRIBUTING.md`). Commit мессежийг `kmod:`, `ui:`, `docs:`, `tests:`, `report:` гэх мэт угтвартай бичсэн.

PR	Гишүүн	Огноо	Агуулга
#1	B242270047	10-22	`/proc/pid` kernel module (`src/kmod/`)
#2	B190910014	10-24	Task hierarchy UI (`src/ui/`)
#3	B190910014	10-24	README Quick Start, CONTRIBUTING, CHANGELOG
#4	B190910014	10-24	kmod: PID-ийг бичигчийн PID namespace-ээр хэвлэх
#5	B241940099	10-25	Хэрэглэгчийн manual, шүүмжийн төлөвлөгөө
#6	B241940099	10-26	Manual: дэмжих орчин, AI тэмдэглэл
#7	B242270047	10-26	Manual: дэмжих орчны засвар
#8	B242270047	10-26	kmod-ийн хэрэгжилтийн тэмдэглэл (`REPORT_KMOD.md`)
#9	B242270047	10-26	Bare-metal тестийн үр дүн, UI screenshot
#10	B242270047	10-26	kmod: төлөвийн тайлбарын засвар
#11	B231910003	10-26	`tests/basic`, `tests/boundary` скриптүүд
#12	B190910014	10-27	`src/prereq/`, Operational manual (§A)
#13	B190910014	10-27	Энэ тайлан ба Manual PDF
#14	B242270047	10-27	Тайлангийн засвар (зураг, шаардлагын хүснэгт, GitHub хэсэг), орчны засвар

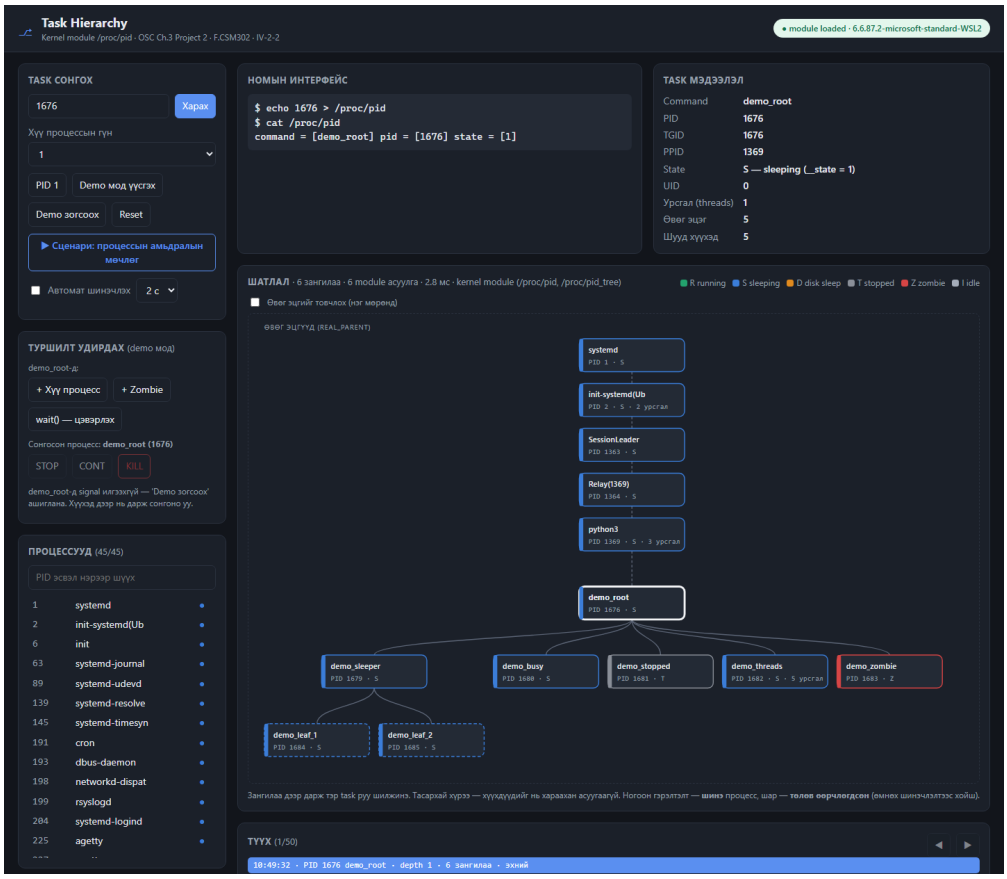
B210930842 нь бэлтгэл module ба Operational manual-ийн ноорогийг (`prereq-manual.docx`) өөрийн fork дээр бэлтгэсэн. B232270108 гүйцэтгээгүй.

5.6 Бэлэн код, сан, AI-ийн хэрэглээ

Kernel module нь сурах бичгийн Programming Project-ийн skeleton (`proc_ops`, `kmalloc`, `copy_from_user`, `kstrtol`, `pid_task`, `find_vpid`) дээр суурилсан. UI-ийн код, Manual, тайлангийн ноорогийг боловсруулахад AI туслах (Claude, Anthropic) ашигласан; бүх кодыг гишүүд уншиж, ажиллуулж, туршиж шалгасан. Гадны программын сан ашиглаагүй (зөвхөн Linux kernel API, Python 3 стандарт сан). Дэлгэрэнгүйг README-ийн “Бэлэн код, сан, AI ашиглалт” хэсгээс харна уу.

6. Туршилт ба UI дүрслэл

Юуг өөрчилж турших	Юу ажиглах
PID (талбар, жагсаалт, зангилаа дээр дарах)	Номын мөр, task мэдээлэл, өвөг эцгийн гинж
Хүү процессын гүн 0–4	PID 1 хүртэл, хүүхдүүд
Demo мод: + Хүү, + Zombie, STOP/CONT/KILL, `wait()`	Модны түвшин, “module асуулга”-ын тоо
Автомат шинэчлэх 1/2/5 с	Шинэ зангилаа ногоон; S→T, S→Z шар; zombie алга болох
▶ Сценари	`demo_busy` R↔S — бодит цагийн төлөв
	fork → SIGSTOP → SIGCONT → SIGTERM (Z) → `wait()`, алхам бүр Түүхэнд
Буруу оролт (`abc`, `-5`, хоосон, хэт том)	Улаан мэдэгдэл, мод цэвэрлэгдэнэ
Module дуудлагын лог	write `/proc/pid` → read `/proc/pid` (байт, µs)



Зураг 2: UI-ийн бүтэц: зүүн талд PID сонголт, туршилт удирдах, процессын жагсаалт; баруун дээд хэсэгт номын интерфейс (`echo`/`cat`) ба task мэдээлэл; доор нь PID 1-ээс `demo_root` хүртэлх өвөг эцгийн гинж ба хүүхдүүд (гүн 1), төлөвийн өнгөний legend. WSL2 хөгжүүлэлтийн орчин.

7. Туршилтын арга

Орчин.

- Bare-metal: Ubuntu 24.04.5 LTS, kernel `7.0.0-34-generic`, gcc 13.3.0, `systemd-detect-virt` = `none`, Secure Boot унтраалттай (`tests/kmod_baremetal.txt`).

- Хөгжүүлэлт: WSL2 Ubuntu 22.04, kernel `6.6.87.2-microsoft-standard-WSL2`, gcc 11.4, `HZ` = 250.

Өгөгдөл ба хүлээгдсэн үр дүн. Туршилтын PID-ийг эх кодод хатуу тогтоогоогүй: скрипт өөрийн shell (`$$`), шинээр үүсгэсэн `sleep`, `pid_max` дотор ашиглагдаагүй дугаар, `ps`-ээс олсон kernel thread-ийг ажиллах үедээ сонгоно.

№	Оролт / нөхцөл	Хүлээгдсэн үр дүн
B1	Ажиллаж буй bash-ийн PID	`command = [bash] pid = [$$] state = [N]`
B2	PID 1	`command = [systemd]`, олон `child` мөр
B3	Байхгүй PID	write амжилттай, read 0 байт
B4	`sleep` хүү үүсгээд эцгийн PID	`child = [sleep] pid = […]`
B5	Kernel thread (`kthreadd`)	мэдээлэл гарна, олон kernel thread хүүхэд
E1–E4	`abc`, `-5`, хоосон, `0`	`write error: Invalid argument`, өмнөх PID хэвээр
E5	`99999999` (> 4194304)	`Invalid argument`
E6	`99999999999999999999`	`Numerical result out of range`
E7	32 байтаас урт мөр	`Invalid argument`
P1–P9	Бэлтгэл module-ууд	dmesg-ийн мөрүүд, `Hello World`, jiffies өснө, seconds ≥ 2

8. Үр дүн

8.1 Тестийн хураангуй

Багц	Bare-metal 7.0.0	WSL2 6.6.87.2
Гараар (`tests/kmod_baremetal.txt`, 10 тохиолдол)	10/10 ✔	—
`tests/basic/test_basic.sh`	5/5 **PASS**	4 **PASS**, 1 SKIP[a]
`tests/boundary/test_boundary.sh`	7/7 **PASS**	7/7 **PASS**
`sudo make -C src/prereq test`	9/9 **PASS**	9/9 **PASS**
`src/ui/tests/test_api.py` (unittest)	24 OK (1 алгассан[b])	24 OK (1 алгассан[b])
`src/ui/tests/test_tree.mjs` (Node)	12/12 (kernel-ээс хамаарахгүй)	
`src/ui/contract_check.py`	17 ✔, 1 ⚠, 0 ✖	17 ✔, 1 ⚠, 0 ✖

a — WSL-ийн PID namespace-д kernel thread харагдахгүй; bare-metal дээр kernel thread тест (B5, PID 2) **PASS**. b — `/proc/pid_tree`-тэй module шаардлагатай тест. ⚠ — `/proc/pid_tree` байхгүй тул `book` горим.

8.2 Bare-metal дээрх бодит гаралт

```
$ sleep 300 &        # pid=5106, parent=5090
$ echo 5090 > /proc/pid; cat /proc/pid
command = [bash] pid = [5090] state = [1]
parent = [bash] ppid = [5084]
child = [sleep] pid = [5106] state = [8193]
child = [cat] pid = [5108] state = [0]
```

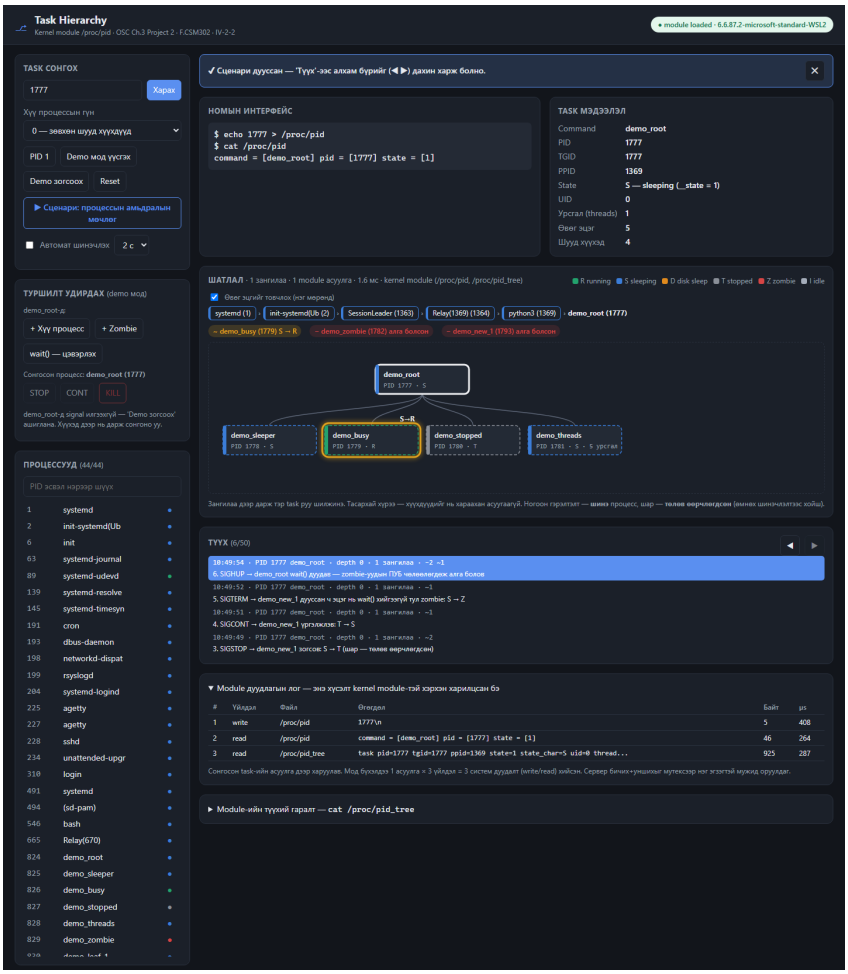
```
$ echo 1 > /proc/pid; cat /proc/pid | head -3        # нийт 33 child
command = [systemd] pid = [1] state = [1]
parent = [swapper/0] ppid = [0]
...
$ echo abc > /proc/pid
bash: echo: write error: Invalid argument
```

`state = [8193]` = `TASK_INTERRUPTIBLE | TASK_FREEZABLE` (унтаж буй `sleep`), `260` = `__TASK_STOPPED | TASK_WAKEKILL` (SIGSTOP).

8.3 Гүйцэтгэл (WSL2, 69 процесс, 20 хэмжилтийн медиан)

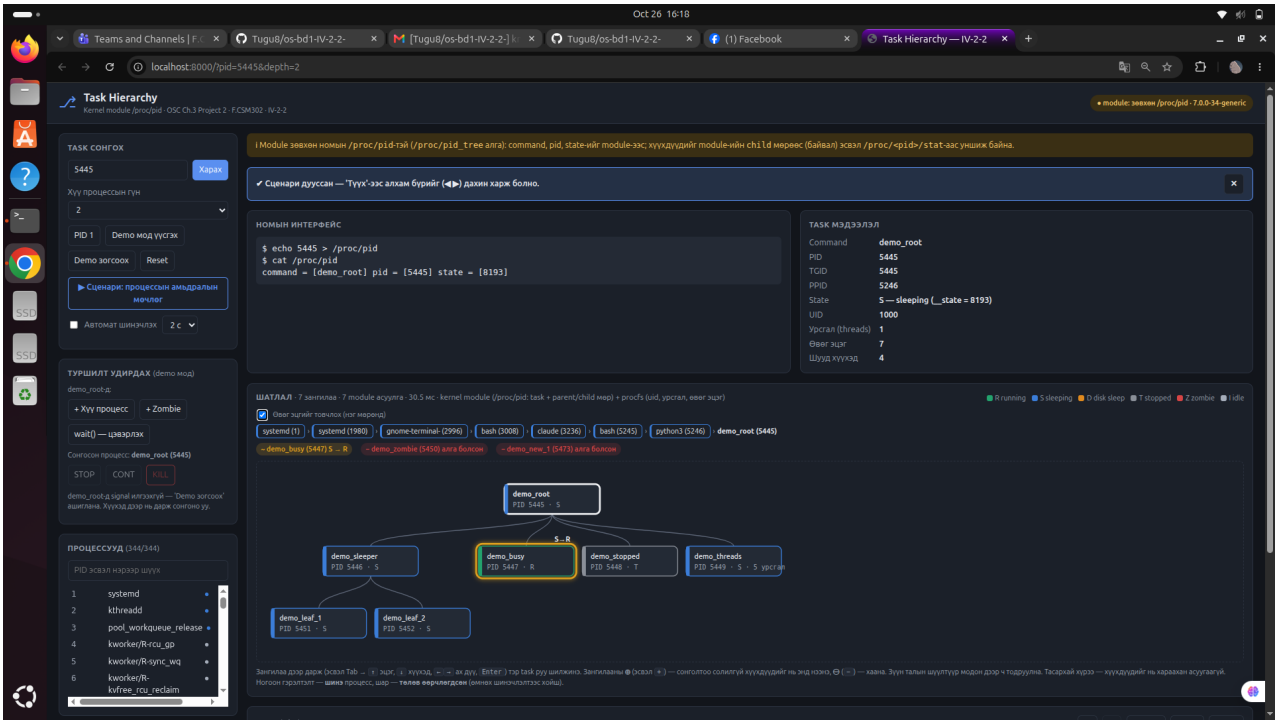
PID 1, гүн	Зангилаа	Module асуулга	Серверийн хугацаа
0	1 (+ 16 навч)	1	5.3 мс
1	17	17	9.8 мс
2	59	59	17.2 мс

Нэг module асуулга: write `/proc/pid` 205 µs, read 850 байт 72 µs. Хугацааны ихэнх нь procfs-оос өвөг эцэг, uid уншихад зарцуулагдана.

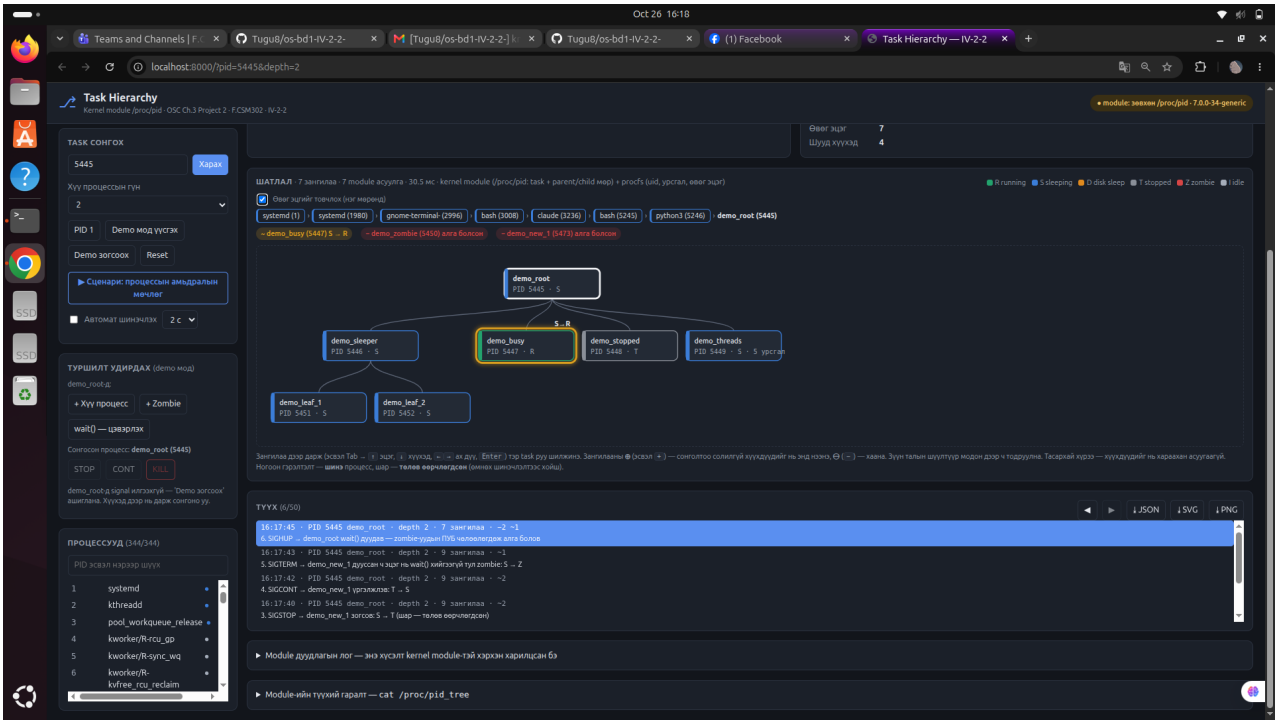


Зураг 3: Сценари дууссаны дараах Түүх (SIGSTOP → SIGCONT → SIGTERM → `wait()`) ба “Module дуудлагын лог”: write `/proc/pid` 5 байт, read 46 байт, тус бүр хэдэн зуун µs. WSL2.

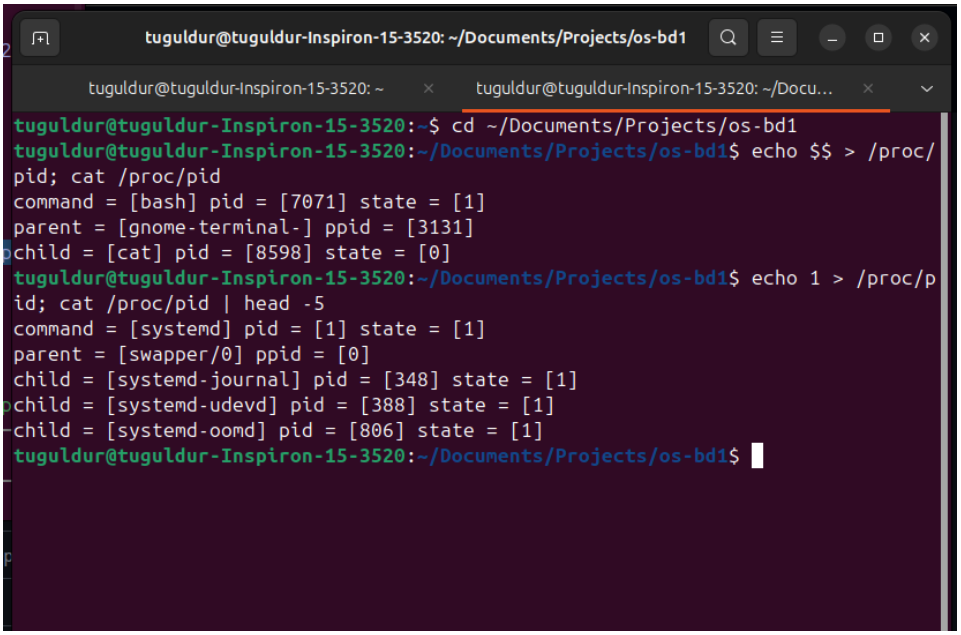
8.4 Screenshot



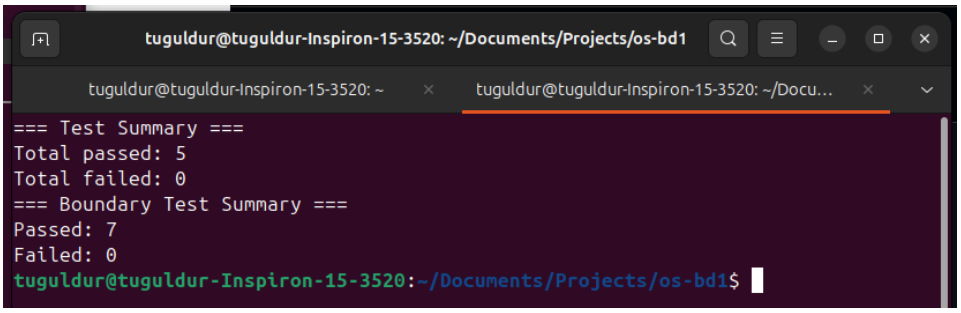
Зураг 4: Bare-metal: `demo_root`-ийн мод, `book` горим, `__state = 8193`.



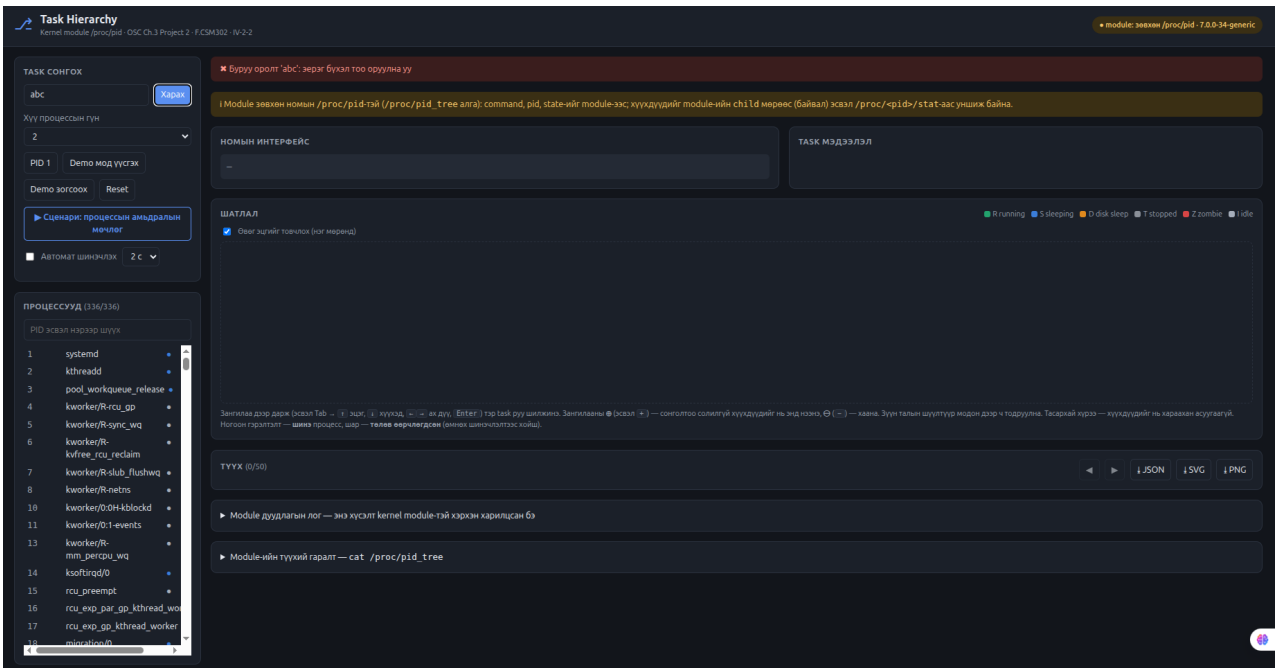
Зураг 5: Bare-metal: сценарийн түүх — SIGSTOP, SIGCONT, SIGTERM → zombie, `wait()`.



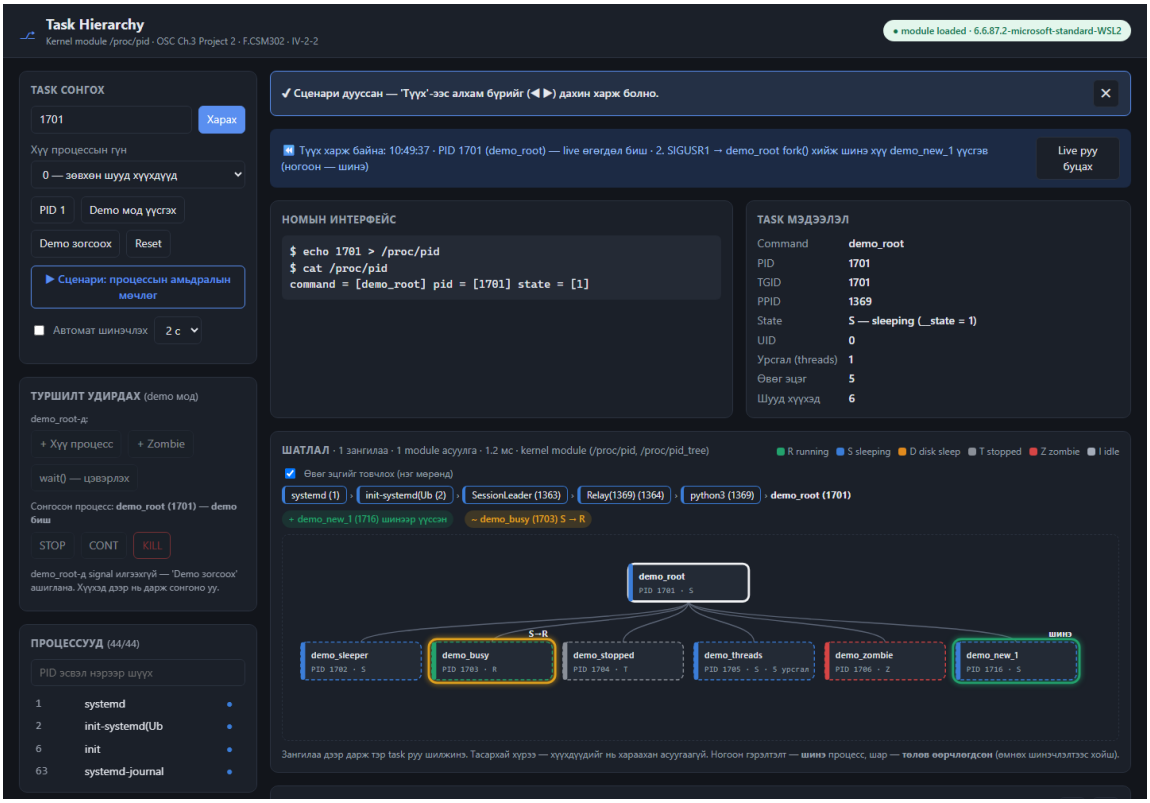
Зураг 6: Bare-metal: номын жишээ — `echo $$ > /proc/pid` (bash, эцэг `gnome-terminal-`) ба `echo 1 > /proc/pid` (systemd).



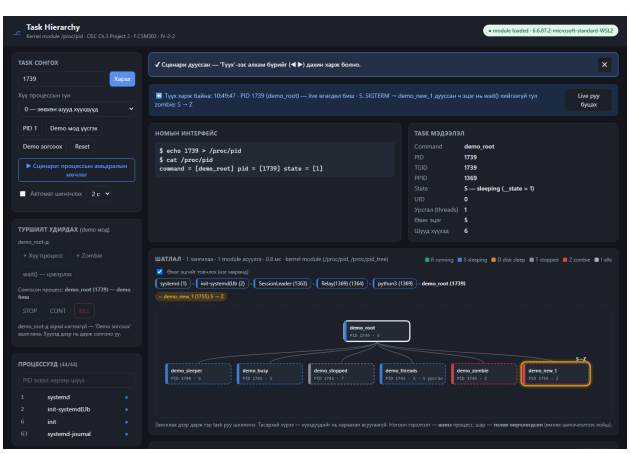
Зураг 7: Bare-metal: `test_basic.sh` 5/5, `test_boundary.sh` 7/7 **PASS** — Summary.

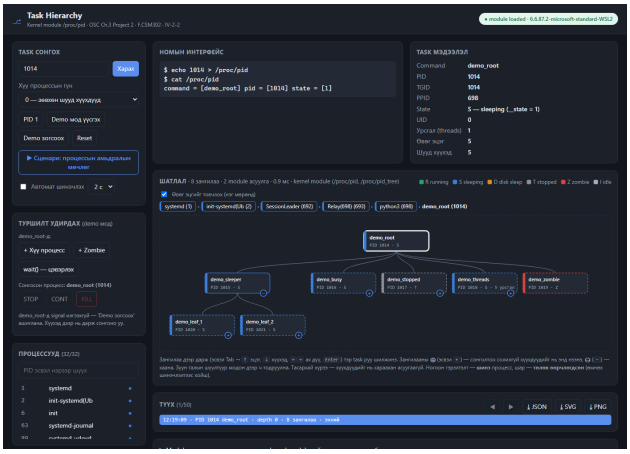


Зураг 8: Bare-metal: UI-д PID талбарт `abc` оруулахад гарах улаан алдааны мэдэгдэл.



Зураг 9: Тест 5: `demo_root fork()` хийж шинэ хүү `demo_new_1` үүсгэв — мод дээр ногоон хүрээтэй “шинэ” гэж тодорч, `demo_busy`-ийн төлөв S→R шараар тэмдэглэгдэв. WSL2.





Зураг 10: Зүүн: SIGTERM → `demo_new_1` S→Z (эцэг `wait()` хийгээгүй). Баруун: газар дээр нь нээх (⊕/⊖), товчилсон өвөг эцэг.

9. Хэлэлцүүлэг

Үр дүнгийн тайлбар. Номын жишээ болон бүх захын оролт хүлээгдсэн үр дүнтэй таарсан; kernel crash, санах ойн алдагдал гараагүй. Bare-metal ба WSL2 хоёр өөр kernel (7.0 ба 6.6) дээр ижил эх код хөрвүүлэгдэж ажилласан нь `proc_ops`, `__state` API-ийн сонголт зөв болохыг харуулна.

Туршилтаар илэрсэн, зассан алдаа.

Асуудал	Засвар
WSL2-д `echo 1` → `pid = [256]`: `task->pid` нь global PID	`task_pid_vnr()` — бичигчийн namespace-ийн PID (`3234d12`)
Хэсэгчлэн (8 байтаар) уншихад мөр тасрах эрсдэл (номын `completed` туг)	`seq_file` + `single_open`
UI-ийн гүн 0 ба 1 ижил мод	`level < depth` → `level <= depth`
Сервер зогсоход туршилтын процессууд өнчин үлдэх	`demo_root` эцгээ шалгаж өөрийгөө цэвэрлэнэ

Хязгаарлалт.

- `l_pid` бүх хэрэглэгчид нэг; терминалаас зэрэг бичвэл бие биеийнхээ PID-ийг дарна (UI мутексээр хамгаалдаг).
- `0666` эрхтэй тул ямар ч хэрэглэгч PID бичнэ; харин `/proc/<pid>/stat`-д аль хэдийн нээлттэй мэдээллийг л харуулна.
- `state` нь битийн маск тул том тоо гарч болно; үсгэн төлөвийг UI procfs-оос авна.
- UI тогтмол хугацаанд асуудаг; хоёр шинэчлэлтийн хооронд үүсээд дууссан процесс харагдахгүй.
- `list_for_each_entry_rcu` нь `children` жагсаалтад хэрэглэгддэг ч албан ёсоор энэ жагсаалтыг `tasklist_lock` хамгаалдаг; маш их ачаалалтай үед ховор зөрүү гарч болзошгүй.
- Kernel < 5.14 дээр хөрвүүлэгдэхгүй; Secure Boot асаалттай бол гарын үсэг шаардана.

10. Дүгнэлт

Сурах бичгийн Programming Project 2-ын бүх шаардлагыг (`/proc/pid` write, read, номын формат, хүчингүй PID → 0 байт) хэрэгжүүлж, task hierarchy дүрслэлд зориулан эцэг, хүүхдийн мөрөөр өргөтгөсөн. 2-р бүлгийн бэлтгэл module-ууд орчныг шалгана. Интерактив UI нь бодит module-тай `/proc/pid`-ээр холбогдож, процессын амьдралын мөчлөгийг (fork, stop, zombie, wait) бодит цагт харуулна. Bare-metal ба WSL2 дээрх 60 гаруй автомат шалгалт, 10 гар тестээр зөв ажиллагааг нотолж, PID namespace-ийн алдааг илрүүлж зассан. Өөр компьютер дээр дахин ажиллуулах алхмыг `docs/Manual.md` (`IV-2-2_Manual.pdf`) болон README-ийн Quick Start-д бичсэн.

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018. — Ch.2–3 Programming Projects, §3.1–3.3, §6.7.

[2] Linux kernel source — `include/linux/sched.h`, `include/linux/proc_fs.h`, `fs/seq_file.c`, `kernel/pid.c`. https://elixir.bootlin.com/linux/latest/source

[3] The Linux Kernel documentation — “The /proc Filesystem”, “seq_file interface”, “RCU concepts”. https://docs.kernel.org/

[4] P. J. Salzman et al. *The Linux Kernel Module Programming Guide*. https://sysprog21.github.io/lkmpg/

[5] F.CSM302 лекц (Г. Ганбат): Lec3 Процесс, Lec4 Урсгал, Lec6 Синхрончлол; OS_BD1_Guide.
[6] Python 3 documentation — `http.server`, `threading`. https://docs.python.org/3/

Хавсралт А. Ажлын төлөвлөгөө

Хуваарилалтын код	IV-2-2
Төсөл	Linux Kernel Module for Task Information
Repository	https://github.com/Tugu8/os-bd1-IV-2-2-
Хэл/сан/орчин	C (Linux kernel module), Python 3 стандарт сан + HTML/CSS/JS; Ubuntu 24.04 bare-metal, WSL2 (хөгжүүлэлт)
Үндсэн дүрслэл	Task hierarchy (parent – task – children мод)
Шүүмжлэх баг	IV-2-5 (Sudoku Solution Validator)

Гишүүн	**Хариуцах ажил**	**Commit/file**
B242270047	Үндсэн kernel module: write, read, parent/children өргөтгөл, `proc_ops`, `__state`; bare-metal тест; Release	`src/kmod/`, PR #1, #7–#10
B190910014	Interactive UI: PID оруулах/сонгох, мод, төлөвийн өнгө, refresh/reset, алдааны мэдэгдэл	`src/ui/`, PR #2–#4
B210930842	2-р бүлгийн бэлтгэл module, build орчин, Operational manual (ноорог)	fork: `prereq-manual.docx`; `src/prereq/`: PR #12
B231910003	Туршилт: номын жишээ, boundary/error тест	`tests/`, PR #11
B232270108	Үндсэн тайлан (Report.pdf). **Гүйцэтгээгүй** — B190910014, B242270047 хийсэн	— (`report/`: PR #13)
B241940099	Хэрэглэгчийн manual, README Quick Start, БД1 пост, IV-2-5-ийн шүүмж	`docs/Manual.md` §B, `review/`, PR #5, #6

Тест	**Оролт/нөхцөл**	**Хүлээгдсэн үр дүн**
1	Ажиллаж буй bash-ийн PID	`command = [bash] pid = [...] state = [...]`
2	PID 1 (systemd/init)	Мэдээлэл гарч, children олон байна
3	Байхгүй PID (жишээ 999999)	Read 0 буцаана, UI ойлгомжтой алдаа харуулна
4	Тоо биш оролт (`abc`), сөрөг тоо	Write алдаа буцаана, kernel crash гарахгүй
5	Child process үүсгээд түүний parent-ийн PID	Мод дээр шинэ child харагдана